

Lab02

Leticia Rocha Soares Martins

2026-08-24

```
install.packages(c("readr", "dplyr"))
```

Installing packages into '/cloud/lib/x86_64-pc-linux-gnu-library/4.6'
(as 'lib' is unspecified)

```
library(readr)  
library(dplyr)
```

Attaching package: 'dplyr'

The following objects are masked from 'package:stats':

filter, lag

The following objects are masked from 'package:base':

intersect, setdiff, setequal, union

Instruções

1) Quais são as estatísticas suficientes para a determinação do percentual de vôos atrasados na chegada (`ARRIVAL_DELAY > 10`)

Resposta -> Para calcularmos o percentual de vôos atrasados precisamos apenas de dois dados: quantidade de vôos com atrasos maior que 10 minutos (`ARRIVAL_DELAY > 10`) e o número total de vôos; Pois o percentual é calculado através da formula:

$$\text{Percentual} = \left(\frac{n_{\text{atrasados}}}{n_{\text{total}}} \right) \times 100$$

2) Crie uma função chamada `getStats` que, para um conjunto de qualquer tamanho de dados provenientes de `flights.csv.zip`, execute as seguintes tarefas (usando apenas verbos do `dplyr`):

- Filtre o conjunto de dados de forma que contenha apenas observações das seguintes Cias. Aéreas: AA, DL, UA e US;
- Remova observações que tenham valores faltantes em campos de interesse;
- Agrupe o conjunto de dados resultante de acordo com: dia, mês e cia. aérea;
- Para cada grupo em b., determine as estatísticas suficientes apontadas no item 1. e os retorne como um objeto da classe `tibble`;
- A função deve receber apenas dois argumentos:
 - `input`: o conjunto de dados (referente ao lote em questão);
 - `pos`: argumento de posicionamento de ponteiro dentro da base de dados. Apesar de existir na função, este argumento não será empregado internamente. É importante observar que, sem a definição deste argumento, será impossível fazer a leitura por partes.

```
# função getStats
getStats = function(input, pos) {
  input |>
    filter(AIRLINE %in% c("AA", "DL", "UA", "US")) |>
    filter(!is.na(ARRIVAL_DELAY), !is.na(YEAR), !is.na(MONTH), !is.na(DAY)) |>
    group_by(YEAR, DAY, MONTH, AIRLINE) |>
    summarise(
      n_atrasados = sum(ARRIVAL_DELAY > 10, na.rm = TRUE),
      n_total = n(),
      .groups = 'drop'
    )
}
```

3) Utilize alguma função `readr::read_***_chunked` para importar o arquivo `flights.csv.zip`.

- Configure o tamanho do lote (chunk) para 100 mil registros;
- Configure a função de callback para instanciar DataFrames aplicando a função `getStats` criada acima;

- Configure o argumento `col_types` de forma que ele leia, diretamente do arquivo, apenas as colunas de interesse (veja nota de aula para identificar como realizar esta tarefa)

```
# especificando as colunas
mycols = cols_only(
  YEAR = 'i',      # i = integer
  MONTH = 'i',
  DAY = 'i',
  AIRLINE = 'c',   # c = character
  ARRIVAL_DELAY = 'd' # d = double
)

# dados separados por chunk de 100.000 observações

dados_voos = read_csv_chunked('flights.csv',
                              callback=DataFrameCallback$new(getStats),
                              chunk_size = 100000,
                              col_types = mycols)

dados_voos
```

```
# A tibble: 1,478 x 6
  YEAR DAY MONTH AIRLINE n_atrasados n_total
  <int> <int> <int> <chr>      <int>   <int>
1  2015     1     1 AA           514    1379
2  2015     1     1 DL           132    1558
3  2015     1     1 UA           367    1269
4  2015     1     1 US           201    1028
5  2015     2     1 AA           649    1508
6  2015     2     1 DL           406    2261
7  2015     2     1 UA           493    1411
8  2015     2     1 US           229    1174
9  2015     3     1 AA           767    1425
10 2015     3     1 DL           702    2021
# i 1,468 more rows
```

4) Crie uma função chamada `computeStats` que:

1. Combine as estatísticas suficientes para compor a métrica final de interesse (percentual de atraso por dia/mês/cia aérea);
2. Retorne as informações em um `tibble` contendo apenas as seguintes colunas:

1. Cia: sigla da companhia aérea;
2. Data: data, no formato AAAA-MM-DD (dica: utilize o comando `as.Date`);
3. Perc: percentual de atraso para aquela cia. aérea e data, apresentado como um número real no intervalo [0,1].

```
computeStats = function(stats) {
  stats |>
    mutate(
      Data = as.Date(paste(2015, MONTH, DAY, sep = "-"))
    ) |>
    mutate(
      Perc = n_atrasados / n_total
    ) |>
    select(
      Cia = AIRLINE,
      Data = Data,
      Perc = Perc
    ) |>
    arrange(Data, Cia)
}
```

```
resultado_final = computeStats(dados_voos)
resultado_final
```

```
# A tibble: 1,478 x 3
  Cia    Data      Perc
  <chr> <date>    <dbl>
1 AA    2015-01-01 0.373
2 DL    2015-01-01 0.0847
3 UA    2015-01-01 0.289
4 US    2015-01-01 0.196
5 AA    2015-01-02 0.430
6 DL    2015-01-02 0.180
7 UA    2015-01-02 0.349
8 US    2015-01-02 0.195
9 AA    2015-01-03 0.538
10 DL   2015-01-03 0.347
# i 1,468 more rows
```

5) Produza um mapa de calor em formato de calendário para cada Cia. Aérea.

1. Instale e carregue os pacotes `ggcal` e `ggplot2`.
2. Defina uma paleta de cores em modo gradiente. Utilize o comando `scale_fill_gradient`. A cor inicial da paleta deve ser `#4575b4` e a cor final, `#d73027`. A paleta deve ser armazenada no objeto `pal`.
3. Crie uma função chamada `baseCalendario` que recebe 2 argumentos a seguir: `stats` (`tibble` com resultados calculados na questão 4) e `cia` (sigla da Cia. Aérea de interesse). A função deverá:
 1. Criar um subconjunto de `stats` de forma a conter informações de atraso e data apenas da Cia. Aérea dada por `cia`.
 2. Para o subconjunto acima, montar a base do calendário, utilizando `ggcal(x, y)`. Nesta notação, `x` representa as datas de interesse e `y`, os percentuais de atraso para as datas descritas em `x`.
 3. Retornar para o usuário a base do calendário criada acima.
4. Executar a função `baseCalendario` para cada uma das Cias. Aéreas e armazenar os resultados, respectivamente, nas variáveis: `CAA`, `CDL`, `cUA` e `cUS`.
5. Para cada uma das Cias. Aéreas, apresente o mapa de calor respectivo utilizando a combinação de camadas do `ggplot2`. Lembre-se de adicionar um título utilizando o comando `ggtitle`. Por exemplo, `cXX + pal + ggtitle("Titulo")`.

```
#=====#
#Não fiz essa ultima questão pois estou usando o posit cloud
#e ele diz que nao tem o pacote "ggcal" disponivel para versão do
#R que estou utilizando (tentei resolver mas nao consegui :/).
#=====#
```

```
Sys.setenv(TZ = "America/Sao_Paulo")
Sys.time()
```

```
[1] "2026-08-24 23:12:14 -03"
```